SmartDB

Household Electrical Fault Detection & Protection System

Full Engineering Development Specification

Field	Value
Document scope	ESP32 edge firmware, cloud AI inference pipeline, Firebase cloud backend, mobile application, hardware procurement, and shared interface contract
Target hardware	ESP32 Developer Kit v1 (edge sensing/control) + cloud AI infrastructure
Software baseline	C/C++ with ESP-IDF (ESP32 firmware), Flutter/Dart (mobile), Firebase (cloud), Python/YOLO + Isolation Forest (cloud AI inference)
Prepared for	SmartDB Project Team
Version	1.1 – API Design Baseline
Date	28 July 2026

> **Design principle:** Detect electrical faults before they become fires. Monitor continuously, alert immediately, cut power automatically when a hazard is confirmed, and never require the homeowner to have technical knowledge to stay safe.

Document Control

Field	Value
Document owner	SmartDB Project — Hardware & Software Engineering
Primary audience	Embedded/firmware engineer, AI/CV engineer, mobile app engineer, cloud/backend engineer, PCB/hardware engineer, QA/test lead
Status	Development baseline. Requirements marked **MUST** are release-blocking unless formally waived.
Product name	SmartDB
Geographic intent	Cambodia, single-household deployment first; architecture should allow multi-home / multi-tenant scaling later
Review cadence	Architecture review at each project phase gate (see Section 24); safety review before field pilot

Revision History

Version	Date	Status	Summary
1.0	28 Jul 2026	Development baseline	Initial complete specification covering edge firmware, AI inference, cloud backend, mobile app, hardware, and shared contracts.
1.1	03 Aug 2026	API Design baseline	Initial API specification and design
1.1.1	04 Aug 2026	Repository-aligned baseline	Reconciled Flutter mobile baseline, implemented gateway, HTTP/OpenAPI collection, operations, verification, and production-gap documentation.

Normative Language

The terms **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** define priority. MUST requirements are mandatory for the targeted release. SHOULD requirements may be deferred only with an explicit engineering decision. MAY requirements are

optional extensions.

> **Important project constraint:** SmartDB is a safety-assist product, not a certified replacement for existing circuit breakers or professional electrical inspection. Release 1 must not claim certified fire-prevention guarantees, must not disable safety features silently, and must always keep a rule-based (non-AI) protection layer active as the fallback.

Contents

1. Executive Summary
2. Product Definition and Scope
3. Reference Architecture
4. **PART A** — Embedded / Edge Engineering (Sensing, ESP32 Firmware, Relay Control)
5. **PART B** — AI Inference Engineering (Jetson Orin Nano, Isolation Forest, YOLO Fusion)
6. **PART C** — Cloud Backend Engineering (Firebase)
7. **PART D** — Mobile Application Engineering (Flutter)
8. **PART E** — Shared Interface, Hardware, Delivery, Risk, and Appendices

Executive Summary

Household electrical faults are a leading cause of residential fires in Cambodia, with **602 serious fire incidents recorded in 2024**, up sharply from 432 the previous year [1]. A local survey of 43 households found that **73.8% had experienced electrical problems**, while **60.5% had no real-time monitoring system** in place, and **over 90% expressed interest** in one.

SmartDB is an IoT-based household electrical monitoring and protection system that detects hazardous conditions — overloads, overheating, current anomalies — before they escalate into fires. The system combines a current sensor, thermal camera, and RGB camera connected to an ESP32-based edge monitoring node, a cloud AI inference layer consisting of electrical anomaly detection and multi-modal decision fusion, a Firebase cloud backend, and a Flutter mobile application.

Key features:

- Real-time monitoring of current and temperature
- Instant mobile alerts when abnormal conditions are detected
- Automatic circuit breaker (relay) activation during critical faults
- Emergency contact access directly through the mobile app
- Historical data logging for review and model recalibration

Release 1 Product Decisions

- **Detection boundary:** Single household circuit per unit in Release 1. Multi-circuit / whole-panel monitoring is a future tier.
- **Protection layer independence:** The relay cutoff decision **MUST** be reachable through a deterministic, rule-based threshold path that does not depend on AI inference availability. AI improves sensitivity and reduces false positives; it is not the sole safety mechanism.
- **Local-first sensing:** Current and thermal readings are always evaluated on-device (ESP32) against hard safety thresholds first, independent of Wi-Fi/cloud connectivity.
- **Offline-capable core safety:** Sensing, threshold evaluation, and relay cutoff continue to function without internet access. Alerts, historical sync, and remote AI inference require connectivity.
- **Human awareness, not full autonomy for edge cases:** Automatic cutoff is reserved for conditions that cross a defined critical safety threshold; borderline/medium-confidence anomalies notify the homeowner rather than cutting power automatically.
- **Deployment baseline:** ESP32 Developer Kit v1 for sensing, local safety decisions, and relay control; cloud AI infrastructure for anomaly detection and decision fusion; Firebase as the backend platform; Flutter mobile app for Android/iOS.

Primary Deliverables

Workstream	Release deliverables
Embedded/Edge (ESP32)	Firmware for sensor acquisition, signal processing, rule-based threshold engine, relay control, secure telemetry transmission, offline buffering.

Workstream	Release deliverables
AI Inference (Cloud)	Isolation Forest anomaly model for current data, YOLO RGB electrical component detector, thermal anomaly analysis, decision fusion engine producing a unified risk score, model retraining pipeline.
Cloud Backend (Firebase)	Realtime database / Firestore schema, authentication, cloud functions for alert routing, historical data storage, device registry.
Mobile App (Flutter)	Live readings dashboard, push alerts with fault classification, historical log viewer, one-tap emergency contact access.
Shared	Telemetry schema, alert/event contract, security model (DTLS/PSK), hardware BOM, test plan, risk register.

Product Definition and Scope

Core Use Case

1. SmartDB's current sensor and thermal camera continuously monitor a household circuit.
2. The ESP32 edge node acquires sensor data on a fixed cycle, extracts features, and evaluates readings against safety thresholds.
3. Telemetry and camera data are sent securely to the cloud AI inference layer, where Isolation Forest evaluates electrical current anomalies, YOLO identifies electrical components from RGB images, and thermal analysis evaluates abnormal heat behavior.
4. A decision fusion engine combines edge safety status, electrical anomaly scores, object detection results, and thermal anomaly scores into a single risk score and confidence band.
5. Low/medium-risk anomalies generate a mobile alert with fault classification. Critical, threshold-crossing conditions trigger **immediate automatic relay cutoff**, independent of AI confirmation.
6. The mobile app shows the homeowner what happened, when, and gives one-tap access to emergency contacts.
7. All readings and events are logged to Firebase for history, review, and future model retraining.

Actors

Actor	Responsibilities / interaction
Homeowner	Views live readings and alerts; responds to notifications; can access emergency contacts; provides feedback used to retrain the anomaly model.
Installer / technician	Mounts sensors, wires the relay into the circuit, performs initial calibration and acceptance test.
Firmware engineer	Builds and maintains ESP32 sensing, threshold logic, relay control, and secure telemetry.
AI/CV engineer	Builds, trains, and validates the Isolation Forest and YOLO models and the fusion engine.
Mobile/backend engineer	Builds the Firebase schema, cloud functions, authentication, and the Flutter app.
QA/test engineer	Validates detection accuracy, response time, and safety-layer independence under simulated fault conditions.

In Scope for Release 1

- One current sensor (SCT-013-000), one thermal camera (MLX90640), one RGB camera module, and one ESP32 edge controller per household unit.
- ESP32-based signal acquisition, RMS current calculation, FFT-based arc detection, and rule-based threshold evaluation.
- Cloud-based Isolation Forest electrical anomaly detection, YOLO RGB electrical component detection, MLX90640 thermal anomaly analysis, and decision fusion producing a unified risk score.
- Automatic relay-based circuit cutoff on critical fault confirmation.
- Firebase-backed live readings, alert history, and event logging.
- Flutter mobile app with live dashboard, alerts, history, and emergency contacts.
- Secure device-to-cloud telemetry (DTLS/PSK, sequence-numbered replay protection).

- Offline core safety behavior (sensing + threshold cutoff continue without connectivity).

Explicitly Out of Scope (Release 1)

- Whole-panel/multi-circuit monitoring (single circuit only in Release 1).
- Remote/manual relay override from the mobile app during a confirmed critical event (safety-first: no user-initiated re-energize without a physical reset in Release 1 — see Open Decisions).
- Integration with third-party smart home platforms (HomeKit, Google Home, etc.).
- Subscription billing/enterprise multi-home fleet management (business-model feature, not Release 1 technical scope).
- Formal electrical safety certification (targeted for a post-pilot phase; see Risk Register R-legal).
- Training a new foundational thermal-detection model from scratch (YOLO fine-tuning on collected data only).

Functional Requirements

ID	Capability	Priority	Requirement
FR-001	Local current sensing	MUST	The ESP32 shall sample current via the SCT-013-000 and compute RMS current on a fixed cycle without interrupting the monitored circuit.
FR-002	Local thermal sensing	MUST	The system shall continuously capture thermal measurements from the MLX90640 and transmit thermal features or frames for cloud-based anomaly analysis.
FR-003	Rule-based safety layer	MUST	A deterministic threshold engine on the ESP32 shall independently evaluate current and temperature against safety limits, without depending on AI inference availability.
FR-004	Automatic relay cutoff	MUST	On a confirmed critical threshold breach, the relay shall disconnect the monitored circuit without requiring user confirmation.
FR-005	AI anomaly detection	MUST	The Isolation Forest model shall flag abnormal current patterns not caught by static thresholds.
FR-006	RGB object detection	MUST	The YOLO model shall identify relevant electrical components from RGB images to provide object context during fault analysis.
FR-007	Thermal anomaly analysis	MUST	The AI service shall analyze MLX90640 thermal features including temperature level, temperature change, and hotspot behavior to identify abnormal heating conditions.
FR-008	Fusion risk scoring	MUST	A fusion engine shall combine rule-based, current-anomaly, and thermal-hotspot signals into a single risk score and confidence band.
FR-009	Mobile alerting	MUST	The mobile app shall receive a push notification within seconds of a confirmed anomaly, including fault classification.
FR-010	Emergency contact access	MUST	The mobile app shall provide one-tap dialing to pre-configured emergency contacts.
FR-011	Historical logging	MUST	Sensor readings, events, and alerts shall be persisted to Firebase for homeowner review and model retraining.
FR-012	Offline core safety	MUST	Sensing, threshold evaluation, and relay cutoff shall continue to operate without internet connectivity.
FR-013	Secure telemetry	MUST	Device-to-cloud communication shall use DTLS with pre-shared keys and sequence-numbered replay protection.
FR-014	Operator feedback loop	SHOULD	The app shall allow the homeowner to confirm/dismiss alerts, feeding a retraining pipeline that reduces false positives over time.
FR-015	Battery backup operation	SHOULD	The device shall continue sensing and alerting from battery power during a mains power outage for a defined minimum duration.

Non-Functional Targets

ID	Quality attribute	Target
NFR-001	Detection accuracy	≥ 90% fault-detection accuracy under simulated fault conditions (pilot target).
NFR-002	Relay response time	Automatic cutoff within an acceptable response window (target: < 1 second from threshold breach confirmation).
NFR-003	Alert latency	Mobile push alert delivered within seconds of anomaly confirmation.
NFR-004	Continuous operation	System operates continuously across a multi-day soak test without unexpected reboot or failure.
NFR-005	Offline duration	Core sensing/protection functions operate on battery backup for a defined minimum duration during a power outage.
NFR-006	False positive rate	Operator feedback loop measurably reduces false-positive rate over successive retraining cycles.
NFR-007	Data isolation	Telemetry transmission uses DTLS/PSK; no plaintext sensor data or credentials on the wire.
NFR-008	Usability	Mobile app rated positively for clarity by test users; alerts understandable without technical/electrical knowledge.

Reference Architecture

SmartDB follows a **five-layer architecture**: Sensing → Edge Processing → AI Inference → Communication → Application.

```
[Current Sensor + Thermal Sensor + Camera]
        │
        ▼
[ESP32 Edge Node]
  - Sensor acquisition
  - RMS calculation
  - Rule-based threshold evaluation
  - Relay control
        │
        ▼
[Cloud AI Inference]
  - Isolation Forest
  - YOLO RGB detection
  - Thermal anomaly analysis
  - Decision fusion
        │
        ▼
[Firebase Cloud Backend]
        │
        ▼
[Flutter Mobile App]
```

Ownership Boundary

Embedded/AI Engineer	Shared Contract	Cloud/Mobile Engineer
Sensor drivers, ESP32 firmware, threshold engine, relay control, Isolation Forest/YOLO models, fusion engine, device security	Telemetry schema, event/alert schema, DTLS/PSK security model, versioning, IDs and timestamps	Firebase schema, cloud functions, authentication, push notification routing, mobile app UI/UX

Data Classification

Data class	Examples	Where stored	Transmission rule	Default retention
Raw sensor telemetry	Current RMS, thermal frame	ESP32 memory → Firebase	DTLS/PSK only	Configurable; suggested 30–90 days detailed
Derived event	Anomaly score, confidence band, fault classification	Firebase	HTTPS/FCM	Retained for history/review
Thermal frame (raw image)	MLX90640 pixel array	Processed on Jetson; not stored long-term by default	Local processing only unless explicitly enabled	Seconds to minutes, destroyed after inference unless flagged as an incident
Homeowner PII	Name, emergency contacts, device ownership	Firebase (encrypted)	HTTPS/TLS only	While account is active
Operational telemetry	Device uptime, WiFi status, battery level	Firebase	HTTPS/MQTT	30–90 days

Shared Engineering Rules

- All timestamps on the wire use UTC ISO 8601. UI converts to local (Asia/Phnom_Penh) time.
- Every telemetry/event message includes `device_id`, `schema_version`, `occurred_at`, and a monotonically increasing `sequence_number` for replay protection. The rule-based threshold path and cloud AI path are **architecturally independent** — a failure or unavailability of cloud services, network connectivity, or AI models must never disable the ESP32's local cutoff capability.
- Relay cutoff commands are logged locally and synced to the cloud once connectivity is restored; they are never silently lost.
- Configuration (thresholds, calibration) is versioned and must be explicitly acknowledged by the device.

PART A — Embedded/Edge Engineering Specification

ESP32 sensing, signal processing, rule-based safety layer, relay control, and security

Embedded/Edge Team Scope

Team Responsibilities

- Select and validate the current sensor, thermal camera, ESP32 board, relay, power/battery subsystem, and enclosure.
- Implement sensor acquisition, RMS current calculation, FFT-based arc/anomaly pre-screening, and I2C thermal capture.
- Implement the deterministic rule-based threshold engine and relay control logic.
- Implement secure telemetry transmission (DTLS/PSK) with sequence-numbered replay protection.
- Implement offline buffering and battery-backed operation during outages.
- Produce the firmware build, hardware test report, and field calibration procedure.

Edge Team Does Not Own

- AI model training/inference (AI team — Part B).
- Firebase schema, cloud functions, and push notification routing (Part C).
- Mobile app UI (Part D).

Firmware Architecture

Repository Layout

```
smartdb-firmware/
├── src/
│   ├── sensors/          # current sensor + thermal camera drivers
│   ├── signal/           # RMS, FFT, feature extraction
```

```
│   ├── safety/           # rule-based threshold engine, relay control
│   ├── comms/            # DTLS/PSK, UDP transmission, telemetry assembly
│   ├── power/            # battery, charging protection, outage handling
│   └── main.cpp
├── lib/
│   └── contracts/        # generated telemetry/event schema headers
├── test/
│   ├── unit/
│   └── hardware-in-loop/
└── docs/
```

Edge Processing Pipeline

```
Sensor acquisition (100ms cycle)
   → Signal processing (RMS calculation, FFT arc-detection pre-check)
   → Level-2 threshold evaluation
   → Telemetry assembly
   → Security layer (DTLS/PSK encryption, sequence numbering)
   → UDP transmission (port 5687) to Jetson/Cloud ingestion
   ↘ (parallel, independent path)
   → Relay cutoff if hard safety limit exceeded
```

Rule-Based Threshold Engine (Independent Safety Layer)

This is the system's non-negotiable safety backstop. It **MUST** function correctly with the Jetson/AI layer, Wi-Fi, and Firebase all unavailable.

- Evaluates RMS current against a configured overload threshold every acquisition cycle.
- Evaluates FFT output for arc-fault–like signatures as a fast pre-screen.
- Evaluates thermal readings (from onboard preliminary processing) against a configured overheat threshold.
- On any hard-limit breach, triggers relay cutoff immediately and logs the event locally.
- Thresholds are loaded from versioned, signed configuration — not hardcoded — so they can be recalibrated without a firmware reflash.

Relay Control

- The relay is driven by a dedicated GPIO path, isolated from the general application logic where feasible, to reduce the chance that a firmware bug in an unrelated module can prevent cutoff.
- Cutoff is a **fail-safe** action: once triggered, the circuit remains open until a deliberate physical reset (see Open Decision OD-01 on remote re-energize policy).
- Every cutoff event is timestamped, logged locally, and queued for cloud sync.

Security

- Telemetry uses **DTLS with pre-shared keys (PSK)**, one key per device, to limit the blast radius of any single compromised credential.
- Every message carries a monotonically increasing `sequence_number`; the receiver rejects replayed or out-of-order messages beyond a tolerance window.
- Command messages (e.g., configuration updates) are authenticated and validated before being applied.

Offline/Battery-Backed Behavior

Function	Offline/outage rule
Current + thermal sensing	Fully available, unaffected by connectivity or mains power loss (battery-backed)
Rule-based threshold evaluation + relay cutoff	Fully available at all times

Function	Offline/outage rule
AI-based anomaly/hotspot detection	Unavailable without Jetson/network reachability; rule-based layer remains the safety net
Telemetry to cloud	Buffered locally; sent once connectivity is restored
Mobile alerts	Delayed until connectivity is restored; local safety action still occurs immediately

Power Subsystem

Component	Purpose
3.7V Li-ion 21700 battery (5000mAh, x4)	Powers ESP32, current sensor interface (ADS1115), and thermal camera during a mains outage
charging/protection module	Charges the battery from the 5V rail; prevents overcharge/overdischarge
Boost converter (3.7V → 5V)	Steps battery voltage up for the AMS1117 regulator input

On a detected outage, the device switches to battery power, continues sensing and local threshold evaluation, and queues telemetry for later sync.

Mandatory Edge Test Cases

Test ID	Scenario	Pass condition
EDGE-SAFE-001	Disconnect network entirely, simulate current overload	Relay cutoff still triggers correctly and within target response time
EDGE-SAFE-002	Simulate firmware crash in non-safety module	Rule-based threshold/relay path unaffected
EDGE-PWR-001	Simulate mains power loss	Device switches to battery, sensing continues, telemetry buffers locally
EDGE-SEC-001	Replay a previously captured telemetry packet	Packet rejected by sequence-number check
EDGE-CAL-001	Apply a signed threshold configuration update	New thresholds take effect without reflash; old config preserved as fallback
EDGE-SIG-001	Inject simulated arc-fault waveform	FFT pre-screen flags the pattern within the decision window
EDGE-I2C-001	Run current sensor (via ADS1115) and thermal camera concurrently	No I2C bus contention or corrupted readings

PART B — Cloud AI Inference Engineering Specification

Isolation Forest, YOLO RGB detection, thermal anomaly analysis, and decision fusion

AI Inference Team Scope

Responsibilities

- Train and validate the Isolation Forest model for electrical current anomaly detection.
- Fine-tune and validate a YOLO model for RGB-based electrical component detection.
- Develop thermal anomaly scoring methods using MLX90640 temperature features.
- Deploy AI inference pipeline on cloud infrastructure.
- Build the fusion engine that combines rule-based, Isolation Forest, and YOLO outputs into a single risk score and confidence band.

- Maintain the retraining pipeline driven by homeowner feedback (confirm/dismiss alerts).
- Produce model cards, validation reports, and known-limitations documentation.

AI Pipeline



Fusion and Decision Rules

- A decision is never made from a single frame/sample; the fusion engine requires agreement across a short decision window (analogous to a multi-sample vote).
- **High confidence** (both electrical anomaly and thermal hotspot agree, or a hard rule-based threshold is independently breached) → immediate mobile alert; relay cutoff already handled by the independent edge safety layer if a hard limit was crossed.
- **Medium confidence** (one signal flags, the other is ambiguous) → mobile alert with a "please check" classification; no automatic cutoff from this path alone.
- **Low confidence / no agreement** → no alert; logged for calibration only.
- Homeowner feedback (confirm/dismiss) is captured and used to retrain the Isolation Forest model, directly targeting the false-positive / alert-fatigue risk (see Risk Register).

Model Governance

Component	Requirement
Isolation Forest	Trained on collected household current-pattern data; versioned; retrained on a defined cadence using homeowner feedback.
YOLO RGB model	Fine-tuned (not trained from scratch) on collected electrical component images; deployed through cloud inference service; model card and validation report required before deployment.
Thermal anomaly model	Uses MLX90640 temperature features to detect abnormal heating behavior.

Component	Requirement
Fusion engine	Deterministic combination logic; thresholds loaded from versioned configuration, not hardcoded.

No model may enter the deployed system without a validation report, version tag, and documented known limitations.

AI/Vision Verification and Acceptance

Test ID	Scenario	Pass condition
AI-DET-001	Simulated overload current pattern	Isolation Forest flags anomaly within target latency
AI-DET-002	Simulated thermal hotspot on test rig	YOLO correctly localizes the hotspot region
AI-FUS-001	Both signals agree on high risk	Fusion engine outputs high-confidence band
AI-FUS-002	Only one signal flags	Fusion engine outputs medium-confidence band, no auto-cutoff from this path
AI-RETRAIN-001	Homeowner dismisses repeated false positives	Retrained model shows measurable false-positive reduction

PART C — Cloud Backend Engineering Specification

Firebase: ingestion, storage, alert routing, device registry

Cloud Backend Team Scope

Responsibilities

- Design the Firestore/Realtime Database schema for devices, readings, events, and homeowner accounts.
- Implement cloud functions for alert validation, deduplication, and push-notification routing (FCM).
- Implement the device registry (device identity, last-seen status, firmware/model versions).
- Implement historical data storage and query APIs for the mobile app.
- Implement authentication (Firebase Auth) for homeowner accounts.

Cloud Team Does Not Own

- Sensor drivers, firmware, or the rule-based safety layer (Part A).
- AI model training/inference (Part B).
- Mobile app UI (Part D).

Core Cloud Data Model

Collection / Table	Purpose
`devices`	Device ID, owner, firmware version, last-seen timestamp, calibration/config version.
`readings`	Time-series current + thermal summary readings per device.
`events`	Anomaly/fault events: confidence band, fault classification, relay-cutoff flag, timestamp.
`feedback`	Homeowner confirm/dismiss responses per event, used for model retraining.
`users`	Homeowner account, emergency contacts, notification preferences.
`audit_log`	Config changes, relay-cutoff commands, calibration updates.

Cloud API/Event Contract

Event Schema (device → cloud, can be modified baed on necessity)

```
{
  "schema_version": "1.0",
  "event_id": "evt_01K...",
  "device_id": "dev_01K...",
  "occurred_at": "2026-07-28T08:12:31.244Z",
  "sequence_number": 18422,
  "event_type": "anomaly.detected",
  "confidence_band": "high",
  "fault_classification": "overheat",
  "relay_cutoff_triggered": true,
  "source": {
    "rule_based": true,
    "isolation_forest_score": 0.92,
    "yolo_hotspot_detected": true
  }
}
```

Cloud Function Responsibilities

- Validate schema and `sequence_number` before persisting (reject replays/duplicates).
- Deduplicate using `event_id` as an idempotency key.
- Route high/medium-confidence events to Firebase Cloud Messaging (FCM) for push notification.
- Update `devices.last_seen` and health status on every telemetry batch.

Cloud Security and Reliability

Area	Requirement
Authentication	Firebase Auth for homeowner accounts; device identity via PSK-derived credentials, not shared secrets in app code.
Data encryption	TLS in transit (HTTPS/FCM); Firestore encryption at rest.
Access control	Firestore security rules scope each homeowner to their own device(s) and data only.
Idempotency	All writes keyed by `event_id`; replaying an event must not create a duplicate record.
Availability target	Cloud ingestion/API availability target ≥ 99.5% during pilot.

Cloud Verification and Acceptance

Test ID	Scenario	Pass condition
CLOUD-ING-001	Duplicate event delivery	Single record persisted
CLOUD-SEC-001	Cross-account data access attempt	Denied by Firestore security rules
CLOUD-ALERT-001	High-confidence event ingested	Push notification delivered within target latency
CLOUD-SYNC-001	Device reconnects after outage with buffered events	All buffered events synced in order, no loss

PART D — Mobile Application Engineering Specification

Flutter app

Mobile App Scope

Responsibilities

- Live dashboard showing current and thermal readings.

- Real-time push alerts with fault classification and confidence band.
- Historical log viewer (past readings and events).
- One-tap emergency contact calling.
- Confirm/dismiss feedback UI feeding the AI retraining loop.

Key Screens

Screen	Content
Dashboard	Live current/temperature readings, device online status, last sync time.
Alerts	Active and past alerts with fault classification, confidence band, and relay-cutoff status.
History	Time-series chart of readings; filterable by date range.
Emergency	One-tap dialing to pre-configured emergency contacts.
Settings	Device management, notification preferences, threshold review (read-only for homeowners).

Alert UX Rules

- Alerts must clearly state what happened, in plain language (e.g., "Unusual heat detected near your outlet — power has been cut off automatically") without requiring electrical knowledge to understand.
- Every alert includes a Confirm/Dismiss action that feeds the model retraining pipeline (FR-014).
- Emergency contact access must be reachable within one tap from any alert screen.

Mobile Verification and Acceptance

Test ID	Scenario	Pass condition
APP-UX-001	Usability test with non-technical users	Users correctly understand and act on an alert without assistance
APP-ALERT-001	High-confidence event pushed	Notification received and displayed within target latency
APP-EMG-001	Tap emergency contact from alert screen	Phone dialer opens with correct number within one tap

PART E — Shared Interface, Delivery, and Program Management

Ownership Matrix (RACI)

Deliverable	Firmware/Edge	AI	Cloud	Mobile	R = Responsible, A = Accountable
Sensor drivers & signal processing	R/A	I	I	I	
Rule-based threshold + relay safety layer	R/A	I	I	I	
Isolation Forest/YOLO models	I	R/A	I	I	
Fusion engine	C	R/A	I	I	
Firestore schema & cloud functions	I	I	R/A	C	
Push notification routing	I	I	R/A	C	
Mobile app UI	I	I	C	R/A	
Telemetry/event schema	R	R	R	C	A: whole team
Security model (DTLS/PSK)	R/A	I	C	I	

Hardware Procurement Summary

Hardware Bill of Materials

Item	Purpose	Cost confidence	Qty	Unit cost	Total
RGB Camera Module	Visual identification of electrical components for AI analysis				
Thermal Camera (MLX90640)	Thermal input	Estimated	1	$56.76	$56.76
ESP32 Developer Kit v1	Firmware and edge processor	Estimated	1	$5.08	$5.08
Current Sensor (SCT-013-000)	Current measurement	Estimated	1	$3.45	$3.45
SmartDB breakout PCB (custom)	Main connecting PCB	Estimated	1	$5–15	~$10
Passive components (PCB build)	Support components	Confirmed	1 set	—	$5–10
Relay (Omron G5LE-1-E)	Emergency power cutoff	Estimated	1	$2.86	$2.86
PCB protection (acrylic conformal coating)	Moisture protection	Estimated	1 can	$12.99	$12.99
Enclosure (custom 3D print, PETG)	Component protection	Estimated	1	$10–20	~$15
Mains wiring (450/750V rated)	Wiring	Estimated	As needed	$0.3–0.5	~$0.4
PG7 cable glands	Cable protection	Estimated	3	$7.99	$7.99
Heat shrink sleeving	Wire protection	Confirmed	As needed	$1.25	$1.25
Ferrules	Wire termination	Estimated	1 set	$9.99	$9.99
Fasteners (M3 standoffs, inserts, screws)	Mounting	Confirmed	1 set	$3.00	$3.00
Boost converter (3.7V → 5V)	Power step-up	Estimated	1	$4.47	$4.47
Battery (3.7V Li-ion 21700, 5000mAh)	Outage backup power	Estimated	4	$5.50	$22.00
Charging protection module (CN3791)	Battery charge protection	Estimated	1	$1.98	$1.98
Total hardware cost					**≈$416**

Software Costs

All software tools used (ESP-IDF/PlatformIO, Flutter, Firebase free tier, Figma, Git/GitHub) are free of charge for the development phase. **Total software cost: $0.**

Software Stack

Technology	Purpose
C++	ESP32 firmware development
Flutter / Dart	Mobile application development
Firebase	Cloud backend and database

Technology	Purpose
Figma	UI/UX design and prototyping
Git/GitHub	Version control and collaboration
Python + YOLO + Isolation Forest	Cloud AI model training and inference

Project Plan and Timeline

Phase	Duration	Window	Exit deliverable
Phase 1 — Research	4 weeks	May–June	Literature review, survey results, defined requirements
Phase 2 — System Design	4 weeks	May–June	Architecture, component selection, DB structure, app workflow
Phase 3 — Hardware Development	2.5 months	July–September	Integrated sensing + relay hardware prototype
Phase 4 — Software Development	2.5 months	July–September	Firmware, mobile app, Firebase integration
Phase 5 — System Integration	2 weeks	September	End-to-end hardware/cloud/app communication established
Phase 6 — Testing & Evaluation	1 month	October	Simulated fault-condition test results
Phase 7 — Analysis & Documentation	2 weeks	October–November	Final report, effectiveness assessment

End-to-End Acceptance Tests

Test ID	Scenario	Pass condition
E2E-001	Simulated overload on live rig	Relay cutoff + mobile alert both occur within target latency
E2E-002	Simulated overheat with network disabled	Relay cutoff still occurs (rule-based path); alert delivered once reconnected
E2E-003	Homeowner dismisses a false-positive alert	Feedback recorded; reflected in next model retraining cycle
E2E-004	Full outage (mains + WiFi) for extended period	Device runs on battery, continues sensing/protection, buffers telemetry, syncs on restore
E2E-005	Emergency contact tap during active alert	Dialer opens correct number in one tap

Evaluation Plan

Success Criteria

Technical performance

- ≥ 90% fault-detection accuracy under simulated conditions
- Relay triggers within target response time
- Mobile alert delivered within seconds of anomaly detection
- Continuous operation without unexpected failure during soak testing

User experience

- Positive usability rating from test users

- Homeowners understand and can respond to alerts without technical knowledge

Data Collection

Type	Data collected
System data	Continuous sensor logs in Firebase during testing; fault-to-cutoff response time; simulated fault-condition results
User data	Pre/post-use surveys on awareness and satisfaction; observed app interaction during testing
Feedback loop	Prototype shared with early users; short questionnaires after each test session; findings presented to stakeholders

Risk Register

ID	Risk	Category	Likelihood	Impact	Risk level
R-01	False negative on fault detection despite fusion of ML and rule-based layers	Technical	Medium	High	High
R-02	DTLS/PSK compromise enabling replay or spoofed override	Technical	Low	High	Medium
R-03	I2C bus contention/noise between ADS1115 and MLX90640	Technical	—	—	—
R-04	PCB isolation failure between mains and low-power domains	Technical	—	—	—
R-05	Firmware bugs causing reboot loops/memory leaks	Technical	—	—	—
R-06	Calibration drift over time requiring threshold readjustment	Technical	—	—	—
R-07	Hand-assembly defects during prototype build	Operational	Medium	Medium	Medium
R-08	Component sourcing delays (import lead times in Cambodia)	Operational	High	Medium	Medium
R-09	Operator alert fatigue from false positives	Operational	Medium	Medium	Medium
R-10	Installation requiring electrical knowledge homeowners lack	Operational	—	—	—
R-11	Enclosure ventilation insufficient in real deployment	Operational	—	—	—
R-12	Network/WiFi outage at deployment site	Operational	—	—	—
R-13	Timeline slippage on AI model training/validation	Project	Medium	High	High
R-14	Insufficient testing time before final demo/launch	Project	—	—	—
R-15	Scope creep from enterprise tier delaying base-tier delivery	Project	—	—	—

ID	Risk	Category	Likelihood	Impact	Risk level
R-16	Single point of knowledge; small team dependency	Project	—	—	—
R-17	Low adoption due to unfamiliarity with proactive electrical safety devices	Market	Medium	High	High
R-18	Price sensitivity in target demographic	Market	—	—	—
R-19	Competition from established smoke-detector brands expanding into smart monitoring	Market	—	—	—
R-20	Difficulty building trust as a new, unproven safety-critical brand	Market	—	—	—
R-21	Component price volatility (Jetson, MLX90640 both imported)	Financial	Medium	Medium	Medium
R-22	Customer acquisition cost exceeding projections	Financial	—	—	—
R-23	Underpriced enterprise tier relative to actual cost	Financial	—	—	—
R-24	Revenue dependent on subscription model facing churn	Financial	—	—	—
R-25	Electrical safety certification requirements not fully mapped for Cambodia	Legal/Regulatory	Low	High	Medium
R-26	Liability exposure if the device fails to prevent a fire it should have detected	Legal/Regulatory	Low	High	Medium
R-27	Data privacy concerns over telemetry revealing home electrical usage patterns	Legal/Regulatory	—	—	—

Mitigation Strategy (High-Priority Risks)

Risk	Mitigation
False negative on fault detection	Rule-based safety layer operates independently of AI inference, ensuring detection continues even if ML models underperform.
DTLS/PSK compromise	Sequence-number-based replay prevention and per-node pre-shared keys limit the blast radius of any single compromised credential.
Hand-assembly defects	Visual inspection and continuity testing performed before powering on each assembled board.
Component sourcing delay	Critical components ordered at project start with a two-week buffer before the assembly phase begins.
Operator alert fatigue	Operator feedback loop retrains the anomaly detection model, reducing false-positive rate over time.
Timeline slippage on AI training	Base-tier model development prioritized ahead of enterprise-tier features to protect the core delivery timeline.
Low market adoption	Pilot deployment with early-adopter households to generate case studies ahead of wider launch.
Component price volatility	Multiple supplier quotes obtained in advance; enterprise-tier pricing includes a margin buffer.

Risk	Mitigation
Electrical safety certification gap	Early consultation with the relevant Cambodian electrical authority to clarify compliance requirements.
Liability exposure	Clear documentation of system limitations; positioned as a complement to, not a replacement for, existing safety equipment.

Stakeholders

Primary Beneficiaries

- **Cambodian homeowners** — real-time monitoring, alerts, and automatic protection.
- **Families in densely populated areas** — early fault detection reduces mass house-fire risk.
- **Elderly or less tech-savvy residents** — automatic cutoff provides protection even when the homeowner is unaware or offline.

Secondary Beneficiaries

- **Fire department** — potential reduction in electrical-fault-caused fire incidents.
- **Local government / municipal authorities** — supports public safety and smart-city initiatives.
- **Electricians and electrical contractors** — may recommend or install the system for homeowners.

Stakeholder Engagement

Feedback is collected through surveys, prototype demonstrations, and user testing, and used to iteratively improve functionality, usability, and effectiveness.

Open Decisions Before Field Pilot

Decision ID	Required decision
OD-01	Remote/manual relay re-energize policy after an automatic cutoff — physical reset only, or an authenticated remote re-energize path?
OD-02	Final current sensor / thermal camera vendor confirmation and CT-clamp installation method for non-technical homeowners.
OD-03	Custom SmartDB breakout PCB design finalization (not yet designed).
OD-04	Enclosure design finalization (not yet designed) — ventilation vs. moisture/dust protection trade-off.
OD-05	Retention policy for raw thermal frames vs. incident-only retention.
OD-06	Electrical safety certification pathway for Cambodia.
OD-07	Battery-backup minimum duration target for Release 1.

Device Ingestion API — Production CoAP/DTLS Contract

This section is the normative ESP32-to-backend contract. It preserves the 100 ms sensing cycle without requiring one network datagram per sample. The device-facing gateway **MUST** expose CoAP over DTLS 1.2 on UDP port **5687**, using a unique per-device PSK. DTLS terminates at the ingestion gateway; only authenticated, validated, deduplicated records cross the trust boundary into AI services and Firebase. Internal forwarding **MUST** use authenticated encryption. The gateway, AI pipeline, and Firebase are never in the local relay-control path.

Versioned Resources and Methods

Resource	Method	Purpose	Default CoAP type	Success
`/v1/telemetry`	POST	One telemetry batch	NON normally; CON for queue drain/final batch	`2.04 Changed`
`/v1/events/critical`	POST	Critical fault/cutoff event	CON	`2.04 Changed`
`/v1/health`	POST	Boot, uptime, queue, radio, and firmware health	NON periodically; CON on boot/fault	`2.04 Changed`
`/v1/config`	GET	Retrieve current signed configuration; use ETag/If-None-Match	CON	`2.05 Content` or `2.03 Valid`
`/v1/config/ack`	POST	Report config validation/application result	CON	`2.04 Changed`

CBOR (`Content-Format: application/cbor`) is mandatory and primary. JSON examples below are explanatory only; production JSON is optional for diagnostics. Unknown optional fields **MUST** be ignored, while missing required fields or unsupported major schema versions are rejected.

Common Envelope and Origin Boundary

Every device-originated POST contains this envelope. `message_id` is a UUIDv4 generated once and persisted with the queued message. `sequence_number` is an unsigned 64-bit, monotonically increasing counter allocated from durable storage before first transmission and never changed on retry. `boot_id` is a random UUID created at boot. Sequence persistence **MUST** use wear-aware checkpointing/reservation blocks so power loss cannot reuse a number.

Field	Type	Required	Rule
`device_id`	string	yes	Provisioned immutable identifier; must match authenticated PSK identity
`schema_version`	string	yes	Semantic version, initially `1.0.0`; major version controls compatibility
`message_id`	UUID string	yes	Stable across retries and duplicate delivery
`sequence_number`	uint64	yes	Persistent, strictly increasing per device
`boot_id`	UUID string	yes	Distinguishes reboot sessions; not a replay reset
`occurred_at`	RFC 3339 UTC string or null	yes	Device event time; null until clock is trustworthy
`time_quality`	enum	yes	`synced`, `estimated`, or `unknown`
`sent_at`	RFC 3339 UTC string or null	yes	Transmission time when known
`payload`	map	yes	Resource-specific device-originated data

Telemetry payload schema: `samples` is a non-empty ordered array; each sample requires `sample_offset_ms`, `current_rms_a`, `max_temperature_c`, and `relay_state`, and may include voltage, FFT/arc features, hotspot features, and local threshold flags. Critical-event payload requires `event_type`, `severity=critical`, `relay_action`, `trigger`, and `local_decision_latency_ms`; it may include a bounded sensor snapshot. Health requires `uptime_s`, `firmware_version`, `queue_depth`, `queue_capacity`, `last_config_version`, and reset reason. Config acknowledgement requires `config_version`, `status` (`applied`, `rejected`, or `deferred`), and optional reason.

Device messages **MUST NOT** contain AI scores, AI classifications, model versions, or fusion results. The backend stores the immutable device record separately, then creates a linked enrichment record containing `source_message_id`, `enriched_at`, `model_version`, scores, classifications, and fusion output. Backend enrichment never overwrites or masquerades as device-originated evidence.

Examples (JSON Rendering of the CBOR Model)

Telemetry batch:

```
{"device_id":"db-001","schema_version":"1.0.0","message_id":"8d8d8d8d-1111-4111-8111-
111111111111","sequence_number":4201,"boot_id":"7a7a7a7a-2222-4222-8222-
222222222222","occurred_at":"2026-07-
28T08:00:00.000Z","time_quality":"synced","sent_at":"2026-07-28T08:00:01.000Z","payload":
{"sample_period_ms":100,"samples":
[{"sample_offset_ms":0,"current_rms_a":4.2,"max_temperature_c":41.3,"relay_state":"closed"}
,
{"sample_offset_ms":100,"current_rms_a":4.3,"max_temperature_c":41.4,"relay_state":"closed"
}]}}
```

Critical event:

```
{"device_id":"db-001","schema_version":"1.0.0","message_id":"9e9e9e9e-3333-4333-8333-
333333333333","sequence_number":4202,"boot_id":"7a7a7a7a-2222-4222-8222-
222222222222","occurred_at":"2026-07-
28T08:00:02.120Z","time_quality":"synced","sent_at":"2026-07-28T08:00:02.150Z","payload":
{"event_type":"overcurrent_cutoff","severity":"critical","relay_action":"opened","trigger":
{"rule":"rms_hard_limit","measured_a":31.2,"limit_a":25.0},"local_decision_latency_ms":180}
}
```

Application acknowledgement (gateway response payload):

```
{"message_id":"9e9e9e9e-3333-4333-8333-
333333333333","sequence_number":4202,"status":"accepted","server_time":"2026-07-
28T08:00:02.180Z"}
```

Config response:

```
{"device_id":"db-001","schema_version":"1.0.0","config_version":17,"issued_at":"2026-07-
28T07:00:00Z","not_before":"2026-07-28T07:00:00Z","expires_at":"2026-08-
28T07:00:00Z","min_firmware":"1.3.0","settings":
{"telemetry_batch_ms":1000,"overcurrent_limit_a":25.0},"key_id":"cfg-2026-
01","signature":"base64url-signature-over-canonical-CBOR-with-signature-omitted"}
```

Config acknowledgement:

```
{"device_id":"db-001","schema_version":"1.0.0","message_id":"abababab-4444-4444-8444-
444444444444","sequence_number":4203,"boot_id":"7a7a7a7a-2222-4222-8222-
222222222222","occurred_at":"2026-07-28T08:00:03Z","time_quality":"synced","sent_at":"2026-
07-28T08:00:03Z","payload":{"config_version":17,"status":"applied"}}
```

Delivery, ACKs, Retry, and Reconnect

A CoAP empty ACK only confirms receipt of a CON datagram; it does **not** mean the record was durably accepted. Success requires a piggybacked or separate `2.xx` application response containing the same `message_id` and `sequence_number`. For a separate response, the device ACKs the server's CON response. NON telemetry has no delivery guarantee and remains queued until covered by an application response from a later CON checkpoint; the checkpoint may acknowledge a contiguous

`accepted_through_sequence` only after durable storage. Critical events remain queued until their explicit application response arrives.

For CON requests, use initial timeout 2 seconds, exponential backoff of 2× with ±25% random jitter, capped at 60 seconds; default maximum is 6 attempts per connection episode. Exact defaults are configurable after network testing. After exhaustion, retain the record and retry on the next connectivity episode. On DTLS timeout/fatal alert, close the session, wait 1 second with jitter, then reconnect with backoff 2, 4, 8, 16, 32, and 60 seconds (cap), resetting only after a stable authenticated exchange. Never busy-loop. A new DTLS session does not change message IDs or sequence numbers.

Offline Queue, Ordering, and Datagram Size

The ESP32 **MUST** use a CRC-protected, power-loss-safe flash ring queue. Priority is: (1) critical events and relay transitions, (2) config acknowledgements and boot/fault health, (3) telemetry, (4) routine health. Send in sequence order within a priority; higher priority may bypass telemetry. Default capacity is **8 MiB or 24 hours of compact telemetry, whichever is smaller**, and must be validated against the selected flash, partition, endurance, and encoded sample size. Reserve at least 10% for priority 1–2. On pressure, evict oldest routine health, then oldest telemetry; never evict an unacknowledged critical event or relay transition. Emit queue-overflow health and counters when possible.

Batch the 100 ms samples into a configurable default 1-second telemetry batch (10 samples). Encoded CoAP message size **MUST** be at most 1024 bytes by default, including CoAP payload but excluding DTLS/IP overhead, and the complete UDP datagram **MUST** remain within the validated path MTU (target ≤1200 bytes) to avoid IP fragmentation. Split batches rather than fragment. CoAP Block-Wise transfer is not used for normal ingestion; images/raw thermal frames require a separately designed upload channel.

Idempotency, Replay, Reboot, and Time

The gateway deduplicates by `(device_id, message_id)` and also enforces uniqueness of `(device_id, sequence_number)`. An exact duplicate returns the original success response; conflicting content for either key returns `4.09 Conflict` and raises a security alert. Maintain a configurable replay window of the highest accepted sequence plus a 1024-sequence bitmap for legitimate reordering. Values below the window are rejected with `4.01 Unauthorized` unless they are known exact duplicates. `boot_id` supports diagnostics only and never resets replay state. If durable sequence state is corrupt or exhausted, the device enters provisioning/recovery mode and does not invent a lower counter.

The device synchronizes UTC using authenticated backend time from successful responses and may also use NTP only after network authentication policy is satisfied. It stores monotonic acquisition offsets regardless of wall-clock state. Large time corrections do not rewrite queued events; backend records receive time and time quality. Configurable defaults: clock is `synced` within ±5 seconds, `estimated` when restored from RTC/checkpoint, and `unknown` otherwise. Sequence/replay validation, not wall time, is authoritative.

Responses and Errors

Code	Meaning / device action
`2.03 Valid`	Cached config remains current
`2.04 Changed`	POST durably accepted or exact duplicate
`2.05 Content`	Config returned
`4.00 Bad Request`	Malformed CBOR/schema; quarantine record and report health
`4.01 Unauthorized`	Identity, replay, revoked key, or authentication failure; stop rapid retries and re-provision
`4.04 Not Found`	Unsupported resource/version; retain critical data and require firmware/operator action
`4.09 Conflict`	ID/sequence reused with different content or config conflict
`4.13 Request Entity Too Large`	Split/re-encode batch

Code	Meaning / device action
`4.15 Unsupported Content-Format`	Use CBOR
`4.29 Too Many Requests`	Honor `Max-Age`/retry hint and back off
`5.00`/`5.03`	Transient backend failure; retain and retry with backoff

PSK Lifecycle, Configuration Trust, and Rate Limits

Each device receives a random, unique PSK of at least 256 bits and a non-secret identity during controlled manufacturing/installation provisioning. The PSK is never logged or placed in source control; store it in ESP32 encrypted NVS protected by Secure Boot and Flash Encryption, preferably backed by a secure element after hardware validation. Backend storage uses a secrets manager/HSM boundary and least privilege. Rotation uses an authenticated, separately wrapped new key with activation time and overlap: device proves possession, stores it atomically, reconnects with the new identity/key, and acknowledges; retain the old key only for a configurable grace period (default 24 hours). Revocation immediately blocks new sessions, terminates active sessions where supported, and requires physical/admin re-provisioning. A compromised PSK cannot authorize config signing.

Config is canonical CBOR, versioned by a strictly increasing `config_version`, and signed by an offline-controlled asymmetric signing key whose public trust anchor is pinned in firmware. Verify device binding, signature, validity interval, minimum firmware, schema, and safe ranges before atomic activation. Persist the highest accepted version; reject rollback even after reboot. Emergency rollback requires a separately authorized recovery key and physical/admin procedure. Invalid config leaves the last known-good safe config active and produces a rejection acknowledgement. Network loss or expired config **MUST NOT** disable local protection; conservative compiled safety limits remain available.

The gateway applies per-device token buckets before expensive processing. Configurable starting defaults are 5 requests/second sustained, burst 20, plus 10 critical events/minute; legitimate critical events are authenticated, durably captured where possible, and security-alerted rather than silently discarded. Return `4.29` with a retry interval. Fleet/global limits and abuse isolation must prevent one device from starving others.

Observability and Safety Invariant

Device metrics include DTLS handshakes/failures, reconnects, request attempts, RTT, response codes, queue depth/high-water mark/evictions, oldest queued age, sequence gaps, clock quality/offset, config version/status, reset reason, and cutoff-to-send latency. Gateway metrics/logs include authenticated device identity, resource, message/sequence IDs, duplicate/replay/conflict decisions, validation result, durable-ingest latency, rate-limit decisions, PSK key version (never key material), and enrichment linkage. Logs must redact secrets and sensitive raw payloads; alerts cover authentication spikes, replay/conflict events, queue saturation, stale devices, and critical-event ingest failures. Correlation uses `device_id`, `message_id`, and `sequence_number` end to end.

Safety invariant: sensor evaluation and relay cutoff execute locally on every 100 ms cycle and achieve the documented target of less than one second from confirmed threshold breach. Cutoff never waits for DTLS, CoAP delivery, an ACK, backend acceptance, AI inference, configuration retrieval, Firebase, or internet availability. A cutoff event is recorded locally and retried until acknowledged, but failure to report it can never delay or reverse the physical cutoff.

Repository Implementation, API Tooling, and Operations

This section records what the repository implements, how its artifacts relate to the normative architecture, and which gaps remain before production. The requirements above remain authoritative; an item described as implemented here is not automatically validated for field deployment.

Repository Map and Sources of Truth

Path	Role	Authority
`docs/architecture_full_v1.1.md`	Product, safety, architecture, and device-ingestion contract	Primary engineering specification

Path	Role	Authority
`backend/`	Python device-ingestion gateway	Executable implementation of the CoAP contract
`postman/specs/smartdb_api/openapi.yaml`	User/admin HTTP API	Normative HTTP interface contract
`postman/collections/smartdb_api/`	Runnable HTTP request examples	Must match the OpenAPI contract
`postman/documentation/README.md`	Postman setup, coverage, security, and maintenance guide	Operational companion to the HTTP contract
`esp_idf/`	ESP32 experiments and camera firmware	Firmware work area; individual projects have separate build configurations
`smart_db/` and `smart_db_v2/`	KiCad source, fabrication output, and design revisions	Hardware work area; revision used for manufacture must be explicitly selected
`docs/reference/`	Background notes, vendor material, and research	Informative, not normative

The HTTP API and device API are intentionally different trust boundaries. Mobile users and administrators use JSON over HTTPS at `/api/v1`. Devices use canonical CBOR over CoAP/DTLS at UDP 5687. The HTTP diagnostic telemetry endpoint must not become a plaintext substitute for production device ingestion.

Postman exercises the HTTP API and therefore uses TLS (`https://`), not DTLS. DTLS is a datagram security protocol for the separate UDP CoAP interface; changing a Postman URL to `coaps://` is not supported and would not test the device contract. Use an ESP32 integration test, a CoAP client with DTLS-PSK support, or a protocol test harness for that interface.

Implemented Device Gateway

The `smartdb-ingestion` Python package currently provides:

- CBOR size enforcement and canonical encoding;
- strict envelope and resource-payload validation, including semantic-version major checks, timezone-aware timestamps, UUIDs, and the full unsigned 64-bit sequence range;
- authenticated identity-to-`device_id` binding at the transport boundary;
- bounded in-memory duplicate and replay tracking for local development;
- uniqueness on `(device_id, message_id)` and `(device_id, sequence_number)`;
- exact retry acceptance, conflicting-content rejection, and replay-window enforcement;
- CBOR application acknowledgements with message ID, sequence, decision, and server UTC;
- `/v1/telemetry`, `/v1/events/critical`, `/v1/health`, `/v1/config/ack`, and `/v1/config`;
- configuration ETags and `2.03 Valid` cache validation; and
- per-identity sliding-window development rate limiting.

The development store retains only message IDs, unsigned 64-bit sequence numbers, and SHA-256 digests inside the configured replay window. It consumes no ESP32 memory because it runs in the Python gateway process, but its state is lost when that process restarts. It therefore cannot satisfy the normative durable-acknowledgement contract by itself. A production Firebase or managed-database adapter must atomically persist the immutable message, both uniqueness keys, and replay state before the gateway returns `2.04`.

Gateway Runbook

Python 3.11 or later is required. From the repository root:

```
cd backend
python -m venv .venv
```

```
. .venv/bin/activate
pip install -e '.[dev]'
SMARTDB_TRANSPORT_MODE=insecure-dev smartdb-ingestion
```

`insecure-dev` is permitted only on a developer-controlled local interface or isolated test network. Secure mode is the default and deliberately refuses startup until a verified server-side DTLS-PSK integration is present. A credential file, when integrated, uses JSON of the form `{"db-001":"<hex-encoded-random-secret>"}` and must remain outside source control. Each production secret must contain at least 256 random bits.

The resource/service layer is transport-independent enough to run behind DTLS, but the current executable does not yet establish a secure DTLS server. aiocoap's DTLS server-side PSK support is experimental and its documented TinyDTLS transport has version-dependent limitations. The preferred production integration is a maintained CoAP/DTLS terminator (for example, a separately validated libcoap- or Californium-based gateway) on the same trusted host/network namespace. It must pass the authenticated PSK identity—not a client-supplied header—to the ingestion service, bind its plaintext upstream to a private local channel, reject all direct plaintext access, and preserve CoAP response codes/options. Using aiocoap's experimental server transport directly is acceptable only after pinning a tested version and proving identity propagation, cipher/version policy, replay behavior, load behavior, and key rotation/revocation in integration tests.

Environment variable	Default	Constraint/purpose
`SMARTDB_HOST`	`0.0.0.0`	UDP bind address; prefer loopback during local testing
`SMARTDB_PORT`	`5687`	UDP 1–65535
`SMARTDB_TRANSPORT_MODE`	`secure`	`secure` or `insecure-dev`
`SMARTDB_DTLS_PSK_FILE`	unset	Required in secure mode; secret JSON path
`SMARTDB_MAX_PAYLOAD_BYTES`	`1024`	Reject larger CBOR payloads
`SMARTDB_REPLAY_WINDOW_SIZE`	`1024`	64–65536 accepted reordering window
`SMARTDB_RATE_LIMIT_PER_MINUTE`	`120`	Development per-identity window limit
`SMARTDB_CONFIG_PATH`	`./config/device-config.cbor`	Offline-signed canonical config

Run automated gateway checks with `cd backend && pytest`. Container builds use `docker build -t smartdb-ingestion backend`; mount read-only `/app/config` and the secret independently. Expose UDP 5687 only after secure transport and an external durable storage adapter are verified.

HTTP API and Postman Collection Rules

The OpenAPI 3.1 document defines authentication, users, devices, desired configuration, telemetry queries, alerts, notifications, and commands. Postman uses `{{base_url}} = http://localhost:8000/api/v1` and variables for device, alert, notification, command, and history ranges. Collection request names and routes use plural REST resources: `/users`, `/devices`, `/alerts`, and `/notifications`.

GET requests do not carry example response objects as request bodies. Mutation methods are POST for creation/actions, PUT for full desired-configuration replacement or alert resolve, and PATCH for partial profile/read-state changes. Success status conventions are `200` for reads/updates with a body, `201` for synchronous creation, `202` for queued/asynchronous work, and `204` when no response body is needed. API implementations must return the shared structured error model and validate bearer authorization on every non-public endpoint.

The Postman collection is an interface client, not proof that an HTTP backend exists. Any future HTTP implementation must be generated from or tested against the OpenAPI document; route changes require updating both artifacts in the same change.

Verification Matrix

Layer	Required command/check	Current evidence	Release gate
Python models/storage	`cd backend && pytest`	Unit tests cover valid ingest, duplicate, conflict, identity, schema, replay, malformed/oversized CBOR, and uint64 maximum	All tests pass
Python package	`pip install -e '.[dev]'`	Console entry point resolves to `smartdb_ingestion.server:main`	Clean environment install succeeds
CoAP local integration	POST CBOR to every resource in `insecure-dev`	Manual/integration test still required	Correct `2.xx`/`4.xx` response matrix
DTLS security	Provision unique PSKs, authenticate, rotate, revoke, replay captured packets	Not implemented in the current aiocoap binding	Mandatory before network exposure
HTTP contract	Validate `postman/specs/smartdb_api/openapi.yaml` with an OpenAPI 3.1 validator	Specification and requests are aligned in-repository	Validator clean and contract tests pass
Firmware	Build the selected ESP-IDF project and run EDGE tests	Multiple experiments exist; release target not yet designated	Reproducible build plus hardware tests
PCB	ERC, DRC, net review, isolation/creepage review, controlled fabrication export	Multiple KiCad revisions exist	Signed manufacturing revision and safety review
End to end	Execute E2E-001 through E2E-004	Pending integrated system	All pass with retained logs

Known Production Gaps and Required Closure

Priority	Gap	Required closure/evidence
P0	Server-side DTLS 1.2 PSK termination is fail-closed but not integrated	Selected supported DTLS terminator, identity binding tests, packet capture showing no plaintext, rotation/revocation test
P0	Hardware and firmware release targets are ambiguous among experiments/revisions	Designate one firmware project and one PCB revision; archive reproducible binaries, fabrication files, ERC/DRC, and review approvals
P0	End-to-end safety timing is unverified	Fault-injection evidence that local cutoff remains below one second without backend/network
P1	Development rate limiter is simpler than the normative token-bucket policy	Implement sustained 5/s burst 20, critical 10/min, fleet isolation, retry metadata, and load/abuse tests
P0	Development replay/deduplication state is memory-only	Add an atomic Firebase/managed-database ingestion adapter; do not issue production `2.04` responses before durable storage
P1	Config file serving does not itself verify signatures	Build offline signing/verification tooling and reject invalid, expired, rollback, wrong-device, or unsafe configurations

Priority	Gap	Required closure/evidence
P1	Gateway observability is minimal	Add secret-safe structured logs, metrics, correlation IDs, security alerts, and durable-ingest latency SLOs
P1	HTTP server implementation is not present in this repository	Implement against OpenAPI, add authentication/authorization, contract tests, database migrations, and audit logging
P2	Generated/build artifacts and backups are mixed with source	Define artifact retention, `.gitignore`, release packaging, and authoritative-source policy without deleting historical engineering data blindly

No production claim, pilot installation, or live mains control is permitted until every P0 gap is closed and the safety review formally accepts the evidence. P1 exceptions require a documented owner, risk acceptance, compensating control, and deadline.

Conclusion

Household electrical faults are a growing safety risk in Cambodia, with reported fire incidents rising from 432 to 602 in a single year. Most homes still rely on traditional circuit breakers that offer no early warning and no real-time visibility. SmartDB addresses this directly through continuous current and thermal monitoring, an independent rule-based safety layer, AI-assisted anomaly detection, and automatic protective cutoff — paired with a mobile app that keeps the homeowner informed without requiring technical knowledge.

Immediate Next Steps

- Complete hardware integration and sensor calibration
- Finalize mobile application development and user testing
- Conduct full system testing under simulated fault conditions
- Present findings and refine the system based on evaluation results
- Explore pathways for wider community adoption beyond the prototype stage

References

[1] Cambodianess, "Fire Surge in Cambodia: 602 Cases, 15 Dead in 8 Months," *Cambodianess*, 2024. Available: https://cambodianess.com/article/fire-surge-in-cambodia-602-cases-15-dead-in-8-months